MSc in Applied Digital Health

Python Bootcamp

Python from scratch for digital health students

Organiser(s): . . . . . Lei Clifton

Instructor(s): . . . . . Bradley Segal, Edward Phillips

Department of Engineering Science
University of Oxford

Michaelmas 2026, revision 1

Table of contents

1 Lab overview 1
2 Getting started 1
2.1 How the notebooks work 2
2.2 When to do what 2
3 Section A: Getting started with Python 3
3.1 Exercise 1: Colab orientation 3
3.2 Exercise 2: Python basics 3
4 Section B: The problem ladder 3
4.1 Exercise 3: Values and variables (Level 1) 3
4.2 Exercise 4: Text and messages (Level 2) 4
4.3 Exercise 5: Decisions (Level 3) 4
4.4 Exercise 6: Lists and loops (Level 4) 4
4.5 Exercise 7: Dictionaries and patient records (Level 5) 4
4.6 Exercise 8: Functions (Level 6) 4
4.7 Exercise 9: Arrays and images (Level 7) 4
4.8 Exercise 10: From a patient record to a model prompt (Level 8) 5
5 Section C: The MedVision laboratory 5
6 Extension: vibe coding a ward dashboard 5
7 After the course 6

Bibliography **7**

1 Lab overview

The goal of this bootcamp is to take you from never having written a line of code to being able to read, write, and verify short pieces of Python code that can interact with patient-style data and medical images. The course culminates in a lab where you will get to run a medical imaging AI model on chest x-rays and evaluate its responses.

AI assistants can now write plausible code in seconds, and some of it is wrong. Someone still has to precisely state what they want, read the output, test it, and decide if it's safe to use. This course teaches you how to be that person and starts off by teaching you the fundamentals of the language.

By the end of this course, you will be able to:

- Run, modify, and save Python notebooks in Google Colab.
- Use values, text, lists, dictionaries, loops, decisions, and functions to work with patient records.
- Read an error message and correct the offending code.
- Think about an image as an array of numbers and prepare it for a model.
- Evaluate a vision-language model on a set of chest X-rays, identify its failure modes, and determine what evidence would be required for clinical use.

The patient and ward records in the course are fictional. The chest X-rays and their short histories are published public domain educational cases. The code is educational code: never use it, or the model from the lab, for patient care.

> **Tip**
>
> Follow the electronic (PDF) version of this sheet so that the links, shown in blue, are clickable.

2 Getting started

Everything is run through Google Colab [1], a free service that enables you to run Python within your web browser. This requires a Google account (personal is fine) and nothing else. There is no software that needs to be installed.

The course materials can be found online at `https://github.com/BradSegal/OxfordADH2026PythonBoo` This page lists every notebook and includes an *Open in Colab* button for each. For each notebook:

1. Press the *Open in Colab* button on the notebook.

2. Choose **File > Save a copy in Drive**. This will open a copy of the notebook in a new tab on your Google Colab. Please work within this new notebook. If you do not do this, your changes will not be saved.

3. Run the first code cell (the setup cell). Colab will give a warning that the notebook "was not authored by Google": Select **Run anyway** as this is from this course. This will install the course helpers (±20 seconds) and print `Course helpers ready`.

4. Work your way down the notebook, running each cell in turn using **Shift+Enter**.

> **Tip**
>
> If a cell complains that a name is not defined, you have probably skipped a cell above it. Choose **Runtime > Run before** to run everything above the current cell, then try again.

2.1 How the notebooks work

The course is a pathway of short notebooks (figure 1). Each level of the problem ladder introduces a new idea and consists of six to seven short problems. The problems are largely of four types [2]: *predict*, where you guess what a piece of code will do before running it; *modify*, where you change existing, working code; *make*, where you create something new; and *fix*, where you solve a bug in a piece of code. Level 8 adds a fifth type, *investigate*, where you read existing code.

Whenever you see `...` or `# Your code here`, replace it with your own code and run the cell. The checker in the cell will give you feedback: correct; not yet, with a hint as to the likely mistake; or not attempted (where the starting code already runs, an untouched problem shows a hint instead). Nothing is marked or recorded, and a checker never blocks the notebook from running. If your own code raises an error, look at the last line, resolve it, then run the cell again. After a fix-the-bug problem there will be a collapsed section (e.g. *Why did that happen?*) that explains it; open it after your own attempt.

All notebooks have a worked version within the `solutions` folder. Make a real attempt first and read the hint. If you are still stuck after ± 10 mins, have a look at the solution for just that one problem, then close it and write the answer yourself.

2.2 When to do what

The day itself consists of a 75-minute lecture and a 75-minute practical. In the practical you will get through `00`, `01`, and Level 1, with most people starting Level 2. You then need to complete the

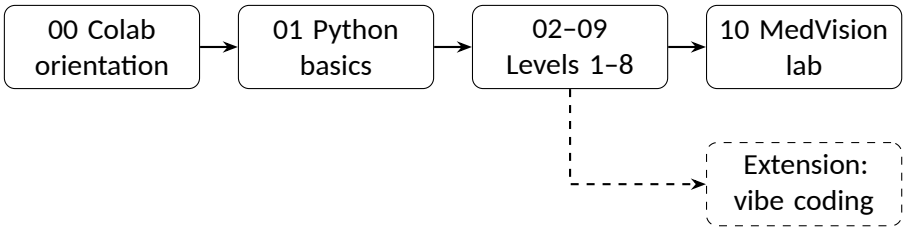


Figure 1: The learning path. Levels 1 to 8 each build on the one before, so do them in order. The extension is optional.

rest at your own pace (a level per sitting) before the start of the AI module on 16 November 2026. First-attempt times are intended as a guide and you can take longer if needed.

3 Section A: Getting started with Python

3.1 Exercise 1: Colab orientation

Notebook `00_colab_orientation` (about 10 minutes). Save your own copy, run cells in order, restart the runtime, and read an error message from its last line upwards. Refer to the two checked tries in the notebook.

3.2 Exercise 2: Python basics

Notebook `01_python_basics` (45 to 60 minutes). A guided ward round in code: variables, lists of observations, a patient record as a dictionary, loops and decisions, and functions as reusable protocols. Each section has a short *Your turn* try. In the practical, stop after the loops section and finish the functions section at home.

4 Section B: The problem ladder

Progress through the different levels in order. Each level begins with a brief recap and ends with a *Before you move on* summary.

4.1 Exercise 3: Values and variables (Level 1)

Notebook `02_L1_values_and_variables` (about 25 minutes). Arithmetic, the order of operations and named values: unit conversions, a weight-based dose, BMI from centimetres, and a running fluid balance. Refer to Problems 1.1 to 1.6.

4.2 Exercise 4: Text and messages (Level 2)

Notebook `03_L2_strings` (about 25 minutes). Strings, f-strings and string methods: patient labels, tidying free text, a standard hospital number, splitting a recorded blood pressure, and converting text to numbers. Refer to Problems 2.1 to 2.6.

4.3 Exercise 5: Decisions (Level 3)

Notebook `04_L3_decisions` (about 30 minutes). Comparisons and `if`, `elif` and `else`: BMI bands, single NEWS2 parameter scores [3], and combining conditions with `and` and `or`. Boundaries matter: check < against <=. Refer to Problems 3.1 to 3.6.

4.4 Exercise 6: Lists and loops (Level 4)

Notebook `05_L4_lists_and_loops` (about 40 minutes). A patient's observations over a day: indexing and slicing, totals and means, counting readings that meet a condition, and finding the first time something happened. Refer to Problems 4.1 to 4.6.

4.5 Exercise 7: Dictionaries and patient records (Level 5)

Notebook `06_L5_dictionaries` (about 30 minutes). Records that pair names with values: updating a record, an allergy check, a synthetic ward as a list of records, and counting by diagnosis. Refer to Problems 5.1 to 5.6.

4.6 Exercise 8: Functions (Level 6)

Notebook `07_L6_functions` (about 40 minutes). Writing a rule once and reusing it: build the NEWS2 score from its parts, assign a risk band, and produce a sorted ward watch list. A missing observation should stop the calculation with a clear error rather than be scored as normal. Refer to Problems 6.1 to 6.6; Problem 6.7 is an optional stretch.

4.7 Exercise 9: Arrays and images (Level 7)

Notebook `08_L7_arrays_and_images` (about 45 minutes). Libraries and numpy arrays: load a chest X-ray as a grid of numbers, crop regions, compare lung zones, and prepare an image the way a model expects it. Refer to Problems 7.1 to 7.6.

4.8 Exercise 10: From a patient record to a model prompt (Level 8)

Notebook `09_L8_bridge_to_medvision` (about 45 minutes). Summarise a patient for a model while sending only the minimum necessary information, build a prompt, and read the library code that the laboratory uses. Refer to Problems 8.1 to 8.6.

5 Section C: The MedVision laboratory

Notebook `10_medvision_lab` (± 75 mins). Prompt a vision-language model (MedGemma [4]) to describe five chest X-rays and compare its responses to each film's reference findings. Iterate on prompt design one change at a time, explore the model's failure modes, and record a small evaluation of your own.

Tip

The laboratory works best with a GPU. In your copy choose **Runtime > Change runtime type**, pick **T4 GPU** and save. The model, about 8.6 GB, downloads the first time and needs no account or token. Colab does not always have a free GPU to give out: without one, the notebook shows replies recorded earlier with a compressed version of the same model, clearly labelled as saved, so you can still answer every question except trying prompts of your own.

Important

The model is a research model that can provide fluent, confident, and incorrect responses. Never use the model or any of the outputs from this course for patient care.

6 Extension: vibe coding a ward dashboard

Notebook `extension/vibe_code_ward_dashboard` (± 75 mins, optional). Build a ward early-warning dashboard using an AI assistant in a disciplined loop. Define the requirements and test cases first, generate code, test and debug it, and then compare two AI-written versions and decide which you would build on and why. It assumes Levels 1 to 6. It does not require an AI assistant, as the notebook provides the code you need to complete each phase.

7 After the course

The notebooks are yours to keep! You have your own copies saved to your Google Drive under *Colab Notebooks*. If you want to start fresh, you can open the notebook again from the course page and save a new copy. The text and exercises are available under CC BY 4.0 and the code under the MIT licence, so feel free to re-use and adapt the content with attribution. If you want a more comprehensive run through the language, the official Python tutorial [5] is a great next step.

Bibliography

[1] Google, "Google Colaboratory: Frequently asked questions," 2026. https://research.google.com/colaboratory/faq.html.

[2] S. Sentance, J. Waite, and M. Kallia, "Teaching computer programming with PRIMM: a sociocultural perspective," *Computer Science Education*, vol. 29, no. 2–3, pp. 136–176, 2019.

[3] Royal College of Physicians, "National Early Warning Score (NEWS) 2: Standardising the assessment of acute-illness severity in the NHS." Updated report of a working party, London: RCP, 2017. https://www.rcp.ac.uk/resources/national-early-warning-score-news-2/.

[4] A. Sellergren *et al.*, "MedGemma technical report." arXiv preprint arXiv:2507.05201, 2025.

[5] Python Software Foundation, "The Python tutorial," 2026. https://docs.python.org/3/tutorial/.